Kevin Küster
Zusammenfassung: Komplexität & Effizienz

Komplexität & Effizienz

Effizienz
Sparsamer Umgang mit Ressourcen während der Programmausführung

Laufzeiteffizienz
Anzahl ausgeführter Operationen

Speichereffizienz
Anzahl benötigter Speicherzellen

} Aufwand in Abhängigkeit zu Problemgröße (n)

Klassifikation des Aufwands

Komplexitätsklassen:

$\log_2(n)$	Logarithmischer Aufwand
n	Linearer Aufwand
$n \cdot \log_2(n)$	Überlogarithmischer Aufwand
n^2	Quadratischer Aufwand
$n^k, k \in \mathbb{N}$	Polynomialer Aufwand
2^n	Exponentialer Aufwand

Quantitativ:

$\log_2 N$	N	$N \log_2 N$	N^2	2^N
0.0	**1**	0.0	1	1
2.32	**5**	11.6	25	32
3.32	**10**	33.2	100	1024
5.64	**50**	282.2	2500	$1*10^{15}$
6.64	**100**	664.4	10000	$1*10^{30}$
8.97	**500**	4482.9	250000	$3*10^{150}$
9.97	**1000**	9965.8	1000000	$1*10^{301}$
12.29	**5000**	61438	$2.5*10^7$	$1*10^{1505}$
13.29	**10000**	132877	$1*10^8$	$2*10^{3010}$

Sortieralgorithmen

Betrachtung: (i) Effizienzermittlung
(ii) Effizienzverbesserung

↳ V: Anzahl Vergleiche

↳ B: Anzahl Bewegungen (Vertauschungen)

⇒ Analyse v. Schleifen & Schachtelungen

Minimumsortieren

```
for i <- 0 to n-2 do              // äußere Schleife
  var m = i
  for j <- i+1 to n-1 do          // innere Schleife
    if  a(j) < a(m)  then m = j   // Index des Minimums finden
  val h = a(i); a(i) = a(m); a(m) = h     // Vertauschen
```

Äußere Schleife: $B = n-1$

Innere Schleife: $V = (n-1) \cdot (n \text{ div } 2)$
(im Mittel: n div 2 Ausführungen)

Sortieren d. Einfügen (Array)

```
for i <- n-2 to 0 by -1 do            // äußere Schleife
  var j = i
  val h = a(i)
  while  j < n-1  &&  h > a(j+1)  do // innere Schleife
    a(j) = a(j+1)
    j += 1
  a(j) = h
```

im Mittel halbe Länge zur Hälfte = n div 4

Äußere Schleife: $B = n-1$

Innere Schleife: $V = (n-1) \cdot (n \text{ div } 4)$

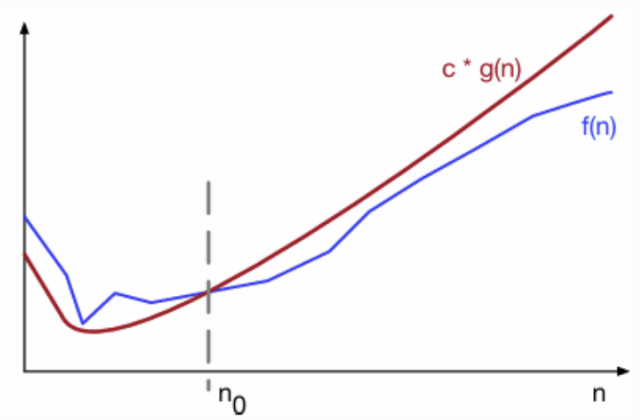
Quicksort

1. Liste in zwei Teillisten auf Basis d. Pivot-Elem
↳ Aufteilung von n Werten ist linear
2. Teillisten rekursiv sortieren
Sofern Listen gleich groß: Teilaufwand $q(\frac{n}{2})$
pro Rekursiver Aufruf $\Rightarrow \log_2(n)$
Insgesamt: $q(n) = n + 2 \cdot q(\frac{n}{2})$
$= n + \log_2(n)$

Asymptotische Analyse

f, g Funktionen, $f(n)$ ist von Ordnung $g(n)$, gdw:
$\exists n_0 \in \mathbb{N}, \exists c \in \mathbb{R} : \forall n \geq n_0 : f(n) \leq c \cdot g(n)$

$\Rightarrow g(n)$ ist Wachstumsrate



Landau - Notation

Landau-Symbol: O

Aufwand $T_A(n) \in \mathbb{R}^+$ eines Algorithmus A bei Datenumfang n ist von Ordnung $g(n)$, gdw: $T_A(n) \in O(g(n))$

$\Rightarrow T_A(n)$ wird von Wachstumsrate $g(n)$ asymptotisch dominiert

Beispiel

$T_A(n) = 2 \cdot n^2 + n \rightarrow T_A(n) \in O(n_2)$

Schätzwerte

Eingabewerte oft undurchsichtig:

↳ **Worst-Case-Analyse**

$T_A(n)$ bezeichnet den größten Aufwand für einen Datenumfang n → Extremfall

↳ **Average-Case-Analyse**

$T_A(n)$ beschreibt den erwarteten durchschnittlichen Aufwand für einen möglichen Datenumfang n unter Berücksichtigung der Häufigkeitsverteilung

↳ O-Notation

Abschätzung nach oben

$\Rightarrow$ Oberschranke für den Aufwand unabhängig von konkreten Werten und Implementierungsdetails

Aufwand: (Worst-Case)

logarithmisch	$T_A(n) \in O(log_2(n))$
linear	$T_A(n) \in O(n)$
überlogarithmisch	$T_A(n) \in O(n \cdot log_2(n))$
quadratisch	$T_A(n) \in O(n^2)$
kubisch	$T_A(n) \in O(n^3)$
polynomial	$T_A(n) \in O(n^k)$
exponentiell	$T_A(n) \in O(2^n)$
faktoriell	$T_A(n) \in O(n!)$

Gesetze

Transitivität

$f(n) \in O(g(n))$, $g(n) \in O(h(n))$

Summenregel

$f_1(n) \in O(g_1(n))$, $f_2(n) \in O(g_2(n))$

$\Rightarrow (f_1(n) + f_2(n)) \in O(max(g_1(n), g_2(n)))$

Anwendung: Falluntorscheidung, wenn Summe zweier Aufwandsfunktionen in Größenordnung des Maximums liegen: $O(n^2+n) = O(n^2)$

$\Rightarrow$ Abschätzung durch größten Term (Transitivität)

Produktregel

$f_1(n) \in O(g_1(n))$, $f_2(n) \in O(g_2(n))$

$\Rightarrow (f_1(n) \cdot f_2(n)) \in O((g_1(n) \cdot g_2(n))$

Anwendung: Verschachtelte Schleifen (Schleifennester)

↳ Sortieralgorithmen

Weitere Funktionsklassen

$f(n) \in O(g(n))$: f wächst höchstens so schnell wie g

$\exists n_0 \in \mathbb{N}, \exists c \in \mathbb{R} : \forall n \geq n_0 : f(n) \leq c \cdot g(n)$

$f(n) \in \Omega(g(n))$: —"— mindestens —"—

$\exists n_0 \in \mathbb{N}, \exists c \in \mathbb{R} : \forall n \geq n_0 : f(n) \geq c \cdot g(n)$

$f(n) \in \Theta(g(n))$: —"— mindestens —"—

$\exists n_0 \in \mathbb{N}, \exists c_1, c_2 \in \mathbb{R} : \forall n \geq n_0 : c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$

$f(n) \in o(g(n))$: f wächst echt langsamer als g

$\exists n_0 \in \mathbb{N}, \exists c \in \mathbb{R} : \forall n \geq n_0 : f(n) < c \cdot g(n)$

$f(n) \in \omega(g(n))$: —"— schneller —"—

$\exists n_0 \in \mathbb{N}, \exists c \in \mathbb{R} : \forall n \geq n_0 : f(n) > c \cdot g(n)$